

MapReduce

Jakub Ciszewski

Maj 2026

Spis treści

1	Opis ćwiczenia	2
2	Środowisko	2
3	Algorytm sekwencyjny	2
3.1	Bezpośrednie mapowanie plików do pamięci (mmap)	3
3.2	Statyczna tablica przeglądowa (Lookup Table)	3
3.3	Bezpośrednie haszowanie potokowe	4
3.4	Niestandardowa płaska tablica mieszająca	4
3.5	Główna funkcja	5
4	Algorytm równoległy	6
4.1	Mapper	6
4.2	Reducer	7
5	Strategia pomiarów	8
5.1	Kompilacja kodu natywnego	8
5.2	Uruchamianie testów wydajnościowych	8
6	Wyniki	9
6.1	Czas wykonania	9
6.2	Przyspieszenie	11
6.3	Metryka COST	11
7	Wnioski	12

1 Opis ćwiczenia

Celem ćwiczenia było porównanie wydajności algorytmu **word count** w wersji sekwencyjnej i rozproszonej z użyciem platformy **Hadoop** w środowisku Amazon EMR.

Badania wydajnościowe przeprowadzono dla zbiorów danych o rozmiarach 1 GB, 10 GB oraz 20 GB. W przypadku infrastruktury rozproszonej przetestowano dwie konfiguracje klastra (wykorzystując wyłącznie węzły typu Core) o stałej łącznej liczbie rdzeni równej 16 vCPU.

Dodatkowo podjęto próbę wyznaczenia metryki **COST** (*Configuration That Outperforms a Single Thread*), określającej minimalną konfigurację sprzętową, dla której system rozproszony osiąga lepszy czas wykonywania niż zoptymalizowany program sekwencyjny.

2 Środowisko

Eksperymenty zostały przeprowadzone w chmurze obliczeniowej Amazon Web Services z wykorzystaniem usługi Amazon EMR (wersja **emr-6.10.0**), dedykowanej do przetwarzania dużych zbiorów danych za pomocą ekosystemu Hadoop.

Wszystkie testy rozproszone uruchomiono w topologii opartej wyłącznie na węzłach typu **Core**. Dla zapewnienia powtarzalności i sprawiedliwości testów, sumaryczna moc obliczeniowa klastra dla obu wariantów wynosiła stale 16 vCPU. Wykorzystano instancje z rodziny **r8g**:

1. **Konfiguracja A (4 węzły r8g.xlarge)**: Każdy węzeł dysponował 4 vCPU oraz 32 GB pamięci RAM, dając łącznie 16 vCPU i 128 GB pamięci operacyjnej.
2. **Konfiguracja B (2 węzły r8g.2xlarge)**: Każdy węzeł wyposażony był w 8 vCPU oraz 64 GB pamięci RAM, co sumarycznie również zapewniało 16 vCPU i 128 GB pamięci RAM.

Węzeł zarządzający (*Primary Node*) w obu przypadkach stanowiła pojedyncza instancja **r8g.xlarge**. Wersja sekwencyjna została uruchomiana na węźle *Primary*.

3 Algorytm sekwencyjny

W celu uzyskania wiarygodnego punktu odniesienia zaimplementowano wysoce zoptymalizowany program sekwencyjny w języku C++. Zamiast standardowych mechanizmów wysokopoziomowych kod opiera się na niskopoziomowych optymalizacjach ukierunkowanych na maksymalne wykorzystanie podsystemu pamięci oraz potokowości procesora.

Poniżej przedstawiono i omówiono kluczowe fragmenty implementacji wraz z zastosowanymi technikami optymalizacyjnymi.

3.1 Bezpośrednie mapowanie plików do pamięci (mmap)

Tradycyjne operacje wejścia/wyjścia (np. `std::ifstream` lub funkcje z rodziny `read()`) wymuszają wielokrotne kopiowanie danych między buforami jądra systemu operacyjnego a przestrzenią użytkownika. W algorytmie zastosowano mapowanie plików bez kopii (`mmap`).

```
89 int fd = open(argv[1], O_RDONLY);
90 // ... [pobranie rozmiaru pliku za pomocą fstat] ...
91 void *addr = mmap(NULL, length, PROT_READ, MAP_PRIVATE, fd, 0);
92 const char *data = static_cast<const char *>(addr);
```

Zastosowana optymalizacja: Wywołanie systemowe `mmap` mapuje zawartość pliku bezpośrednio do przestrzeni adresowej procesu. Dzięki temu algorytm uzyskuje dostęp do danych za pomocą zwykłego wskaźnika tekstowego (`const char*`), eliminując narzut na alokację buforów oraz operacje I/O, co pozwala na przetwarzanie danych z prędkością ograniczoną niemal wyłącznie przez przepustowość pamięci RAM.

3.2 Statyczna tablica przeglądowa (Lookup Table)

Podczas ekstrakcji słów kluczowa jest szybka weryfikacja, czy dany bajt reprezentuje znak alfanumeryczny. Zamiast kosztownych instrukcji warunkowych lub wywołań funkcji bibliotecznych typu `isalnum()`, zastosowano tablicę przeglądową wygenerowaną na etapie kompilacji.

```
23 struct CharLut {
24     bool data[256]{};
25     constexpr bool operator[](size_t idx) const { return data[idx]; }
26 };
27
28 constexpr CharLut generate_word_lut() {
29     CharLut lut{};
30     for (int i = 0; i < 256; ++i) {
31         if ((i >= 'a' && i <= 'z') || (i >= 'A' && i <= 'Z') || (i >= '0' && i <= '9')) {
32             lut.data[i] = true;
33         }
34     }
35     return lut;
36 }
37
38 alignas(64) constexpr CharLut is_word_char = generate_word_lut();
```

Zastosowana optymalizacja: Słowa kluczowe `constexpr` wymuszają, aby tablica `is_word_char` o rozmiarze 256 bajtów została wypełniona w trakcie kompilacji. Dodatkowo dyrektywa `alignas(64)` wyrównuje strukturę do granicy linii pamięci podręcznej (*cache line*). Sprawdzenie znaku sprowadza się do pojedynczego odczytu z pamięci $O(1)$ pod indeksem będącym wartością bajtu,

całkowicie eliminując skoki warunkowe i ryzyko błędnego przewidywania skoków (*branch misprediction*) przez procesor.

3.3 Bezpośrednie haszowanie potokowe

Do generowania indeksów w tablicy mieszającej zaimplementowano uproszczoną, zoptymalizowaną wersję algorytmu haszującego FNV-1a.

```
9 inline uint64_t hash_fast(const char *str, size_t len) {
10     uint64_t hash = 0xcbf29ce484222325;
11     for (size_t i = 0; i < len; ++i) {
12         hash ^= str[i];
13         hash *= 0x000001000000001B3;
14     }
15     return hash;
16 }
```

Zastosowana optymalizacja: Słowo kluczowe `inline` sugeruje kompilatorowi rozwinięcie funkcji w miejscu wywołania. Sam algorytm operuje wyłącznie na rejestrach procesora za pomocą tanich operacji bitowych (XOR) oraz mnożenia liczb całkowitych, co pozwala na błyskawiczne generowanie unikalnych sygnatur dla odnalezionych ciągów znaków.

3.4 Niestandardowa płaska tablica mieszająca

Wykorzystanie struktur takich jak `std::map` czy `std::unordered_map` w C++ wiąże się z częstą alokacją pamięci na stercie i rozproszeniem wskaźników. W programie zaimplementowano autorską strukturę typu *Flat Hash Map* z otwartym adresowaniem i liniowym próbkowaniem.

```
40 struct WordSlot {
41     const char *start = nullptr;
42     uint32_t len = 0;
43     uint32_t count = 0;
44 };
45
46 void word_count_ultra(const char *data, size_t size) {
47     const size_t MAP_SIZE = 1 << 22; // 2^22
48     const size_t MASK = MAP_SIZE - 1;
49     std::vector<WordSlot> hash_map(MAP_SIZE);
50     // ...
```

Zastosowana optymalizacja:

- **Brak kopii:** Struktura `WordSlot` nie przechowuje ciągów tekstowych typu `std::string`, a jedynie wskaźnik na początek słowa w zmapowanym pliku (`const char*`) oraz jego długość. Chroni to aplikację przed alokacjami pamięci podczas dodawania nowych słów.

- **Przyjazność dla pamięci podręcznej (Cache-friendly):** Tablica alokowana jest jako jeden ciągły blok pamięci. Rozmiar mapy dobrano jako potęgę dwójki ($1 \ll 22$), co umożliwia zastąpienie kosztownej operacji modulo % na operację logicznego AND za pomocą maski bitowej (`hash & MASK`). Rozwiązywanie kolizji poprzez liniowe przeszukiwanie sąsiednich komórek pamięci maksymalizuje wykorzystanie mechanizmu *hardware prefetcher*a procesora.

3.5 Główna funkcja

```

1 void word_count_ultra(const char *data, size_t size) {
2     const size_t MAP_SIZE = 1 << 22;
3     const size_t MASK = MAP_SIZE - 1;
4     std::vector<WordSlot> hash_map(MAP_SIZE);
5
6     const char *ptr = data;
7     const char *end = data + size;
8     size_t unique_words = 0;
9
10    while (ptr < end) {
11        while (ptr < end && !is_word_char[static_cast<unsigned char>(*ptr)]) {
12            ptr++;
13        }
14        if (ptr >= end) break;
15
16        const char *word_start = ptr;
17        while (ptr < end && is_word_char[static_cast<unsigned char>(*ptr)]) {
18            ptr++;
19        }
20        size_t word_len = ptr - word_start;
21
22        uint64_t hash = hash_fast(word_start, word_len);
23        size_t idx = hash & MASK;
24
25        while (true) {
26            if (hash_map[idx].count == 0) {
27                hash_map[idx] = { word_start, static_cast<uint32_t>(word_len), 1 };
28                unique_words++;
29                break;
30            }
31            if (hash_map[idx].len == word_len &&
32                std::memcmp(hash_map[idx].start, word_start, word_len) == 0) {
33                hash_map[idx].count++;
34                break;
35            }
36            idx = (idx + 1) & MASK;

```

```

37     }
38 }
39 std::cout << "Przetwarzanie zakonczone. Unikalnych slow: " << unique_words << "\n";
40 }

```

4 Algorytm równoległy

Algorytm równoległy zaimplementowano również w języku C++ w celu osiągnięcia jak najbardziej sprawiedliwych wyników. Wersja równoległa składa się z dwóch algorytmów: *mapper* i *reducer*.

4.1 Mapper

```

1  #include <iostream>
2  #include <string>
3
4  struct CharLut {
5      bool data[256]{};
6      constexpr bool operator[](size_t idx) const { return data[idx]; }
7  };
8
9  constexpr CharLut generate_word_lut() {
10     CharLut lut{};
11     for (int i = 0; i < 256; ++i) {
12         if ((i >= 'a' && i <= 'z') || (i >= 'A' && i <= 'Z') || (i >= '0' && i <= '9')) {
13             lut.data[i] = true;
14         }
15     }
16     return lut;
17 }
18
19 alignas(64) const CharLut is_word_char = generate_word_lut();
20
21 int main() {
22     std::ios_base::sync_with_stdio(false);
23     std::cin.tie(NULL);
24
25     std::string line;
26     while (std::getline(std::cin, line)) {
27         const char *ptr = line.data();
28         const char *end = ptr + line.size();
29
30         while (ptr < end) {
31             while (ptr < end && !is_word_char[static_cast<unsigned char>(*ptr)]) ptr++;
32             if (ptr >= end) break;
33

```

```

34         const char *word_start = ptr;
35         while (ptr < end && is_word_char[static_cast<unsigned char>(*ptr)]) ptr++;
36
37         std::cout.write(word_start, ptr - word_start);
38         std::cout << "\t1\n";
39     }
40 }
41 return 0;
42 }

```

4.2 Reducer

```

1  #include <iostream>
2  #include <string>
3  #include <cstdlib>
4
5  int main() {
6      std::ios_base::sync_with_stdio(false);
7      std::cin.tie(NULL);
8
9      std::string line;
10     std::string current_word = "";
11     uint64_t current_count = 0;
12
13     while (std::getline(std::cin, line)) {
14         size_t tab_pos = line.find('\t');
15         if (tab_pos == std::string::npos) continue;
16
17         std::string word = line.substr(0, tab_pos);
18         uint64_t count = std::stoull(line.substr(tab_pos + 1));
19
20         if (current_word == word) {
21             current_count += count;
22         }
23         else {
24             if (!current_word.empty()) {
25                 std::cout << current_word << "\t" << current_count << "\n";
26             }
27             current_word = word;
28             current_count = count;
29         }
30     }
31
32     if (!current_word.empty()) {
33         std::cout << current_word << "\t" << current_count << "\n";
34     }
35 }

```

```
36     return 0;
37 }
```

5 Strategia pomiarów

W celu zachowania rzetelności metodologicznej, każdy eksperyment pomiarowy dla obu wariantów architektonicznych (sekwencyjnego oraz rozproszonego) został przeprowadzony dwukrotnie, a do ostatecznej analizy przyjęto wyliczoną średnią arytmetyczną wraz z wariancją.

5.1 Kompilacja kodu natywnego

Natywny kod źródłowy algorytmu sekwencyjnego oraz binarne komponenty środowiska rozproszonego zostały skompilowane za pomocą kompilatora `g++` z zachowaniem identycznych flag optymalizacyjnych:

```
g++ -O3 -march=native -std=c++17 program.cpp -o program
```

Flaga `-O3` aktywowała agresywne optymalizacje kodu, natomiast `-march=native` pozwoliła na automatyczne dostosowanie instrukcji procesora do architektury układów.

5.2 Uruchamianie testów wydajnościowych

- **Wariant sekwencyjny:** Program uruchamiano bezpośrednio z poziomu powłoki Bash na węźle *Primary*, przekazując ścieżkę do plików testowych umieszczonych na wolumenie `/mnt1`. Czas mierzony był wewnętrznie przez aplikację z użyciem biblioteki `<chrono>`.
- **Wariant rozproszony (Hadoop):** Pliki wejściowe (1 GB, 10 GB, 20 GB) zostały uprzednio załadowane do rozproszonego systemu plików HDFS. Zadania mapowania i redukcji uruchamiano poprzez moduł Hadoop Streaming za pomocą poniższego szablonu komendy:

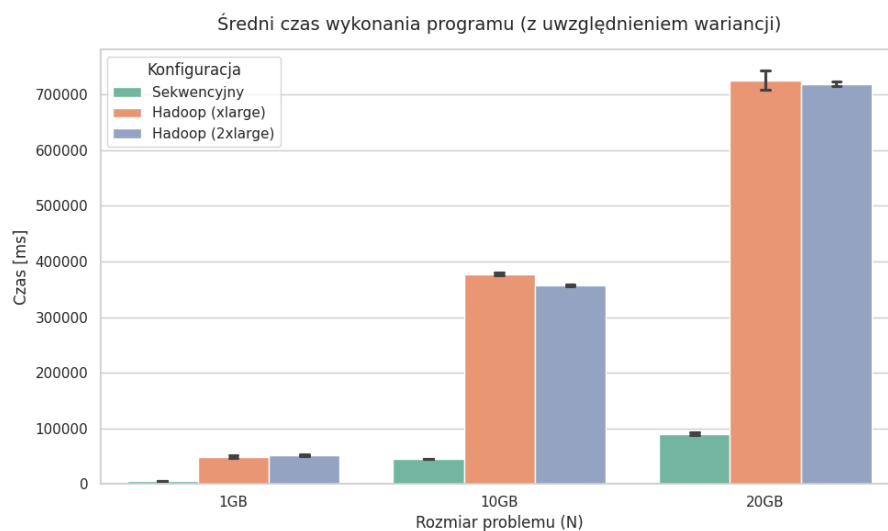
```
time hadoop jar /usr/lib/hadoop/hadoop-streaming.jar \  
-files mapper,reducer \  
-mapper ./mapper \  
-reducer ./reducer \  
-input /wordcount/katalog_rozmiaru/ \  
-output /output_rozmiaru
```

Do końcowych zestawień wydajnościowych pobierano czas `real` zwracany przez systemowe polecenie `time`. Przed każdym kolejnym uruchomieniem katalog wyjściowy `-output` w systemie HDFS był usuwany.

6 Wyniki

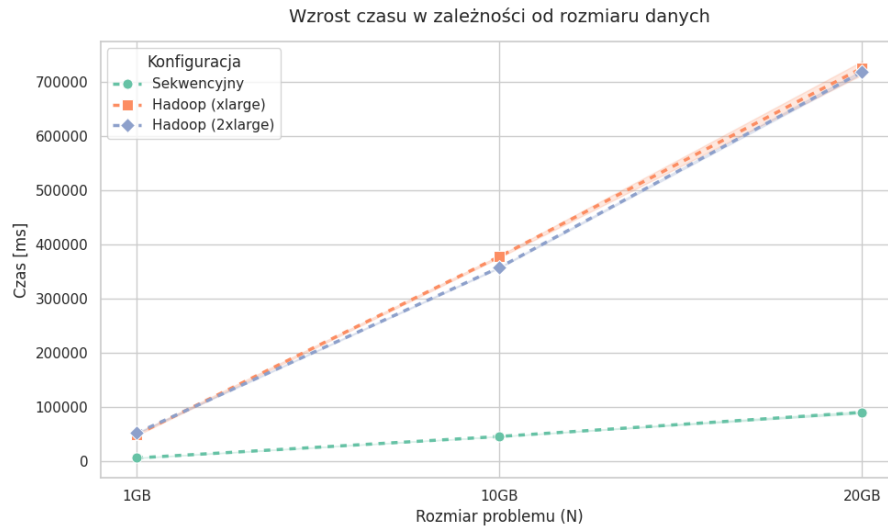
6.1 Czas wykonania

Na rysunku 1 możemy zaobserwować, że algorytm sekwencyjny osiąga znacznie niższe czasy wykonania dla każdego rozmiaru problemu, jednocześnie będąc też bardziej stabilnym czasowo, co sugeruje małe odchylenie standardowe.



Rysunek 1: Średni czas obliczeń z odchyleniem standardowym w zależności od rozmiaru problemu i konfiguracji

Zdecydowanie większe czasy dla wariantu równoległego są spowodowane dużym narzutem startowym oraz konfiguracyjnym platformy Apache Hadoop. Dla testowanych wolumenów danych (1–20 GB) narzut ten dominuje nad samą fazą czystych obliczeń.

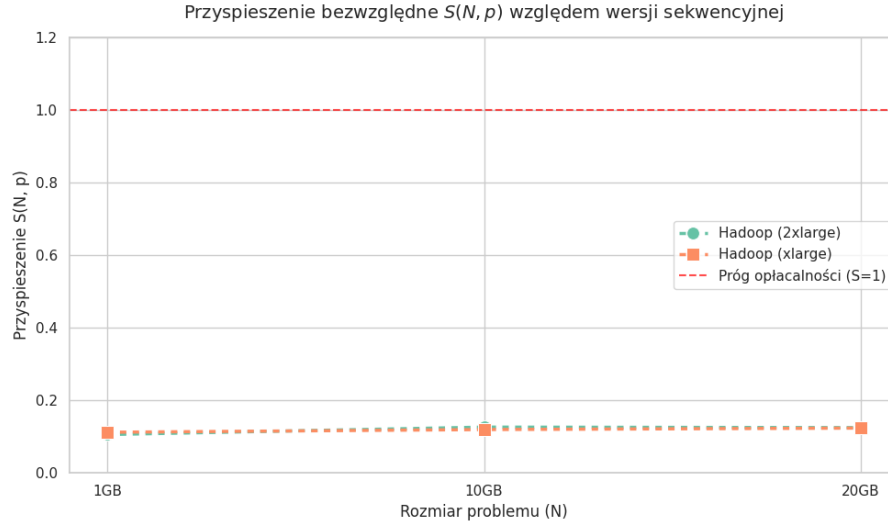


Rysunek 2: Czas wykonania w zależności od rozmiaru problemu i konfiguracji

Na rysunku 2 możemy zauważyć, że dla wszystkich testowanych wariantów wzrost czasu wykonania ma charakter liniowy. Jednakże współczynnik kierunkowy prostej reprezentującej konfigurację rozproszoną jest drastycznie większy niż w przypadku algorytmu sekwencyjnego. Dowodzi to, że narzut środowiska rozproszonego platformy Hadoop nie ogranicza się wyłącznie do stałego kosztu startowego klastra.

Porównując obie konfiguracje klastra, podejście zorientowane na skalowanie pionowe (Konfiguracja B: 2 węzły `r8g.2xlarge`) okazało się nieznacznie efektywniejsze dla większych zbiorów danych (10 GB oraz 20 GB) niż skalowanie poziome (Konfiguracja A: 4 węzły `r8g.xlarge`). Wynika to z mniejszej intensywności fazy *Shuffle*, w której dane muszą być przesyłane przez sieć między fizycznymi instancjami. Mniejsza liczba większych węzłów redukuje narzut na komunikację sieciową.

6.2 Przyspieszenie



Rysunek 3: Przyspieszenie w zależności od konfiguracji i rozmiaru problemu

W celu dokonania oceny efektywności wariantu równoległego, wyznaczono współczynnik przyspieszenia bezwzględnego $S(N, p)$ zgodnie ze wzorem:

$$S(N, p) = \frac{S_k(N)}{T(N, p)} \quad (1)$$

gdzie $S_k(N)$ reprezentuje średni czas wykonania zoptymalizowanego algorytmu sekwencyjnego dla rozmiaru problemu N , natomiast $T(N, p)$ odpowiada średniemu czasowi uzyskanemu w środowisku rozproszonym.

Na rysunku 3 wyraźnie widać, że dla obu konfiguracji klastra Hadoop przyspieszenie przyjmuje wartości skrajnie niskie, oscylujące w granicach $S(N, p) \approx 0.11 - 0.13$. Wyniki te plasują się daleko poniżej czerwonej linii oznaczającej próg opłacalności stosowania systemów równoległych ($S = 1$). Świadczy to o tym, że dla badanych wielkości plików (1–20 GB) narzut na koordynację klastra, operacje dyskowe HDFS oraz komunikację siecią całkowicie niweluje zysk z przetwarzania danych na 16 vCPU.

6.3 Metryka COST

Metryka COST (*Configuration That Outperforms a Single Thread*) określa minimalną konfigurację sprzętową systemu rozproszonego, jaka jest wymagana, by zrównać się wydajnością z poprawnie zaimplementowanym programem jednowątkowym.

W badanym eksperymencie, dla zbiorów danych o rozmiarach od 1 GB do 20 GB, metryka COST przyjmuje wartość **nieograniczoną** (*unbounded COST*). Wynika to z faktu, że czas wykonania klastra składającego się z 16 vCPU był w każdym przypadku od 7 do 9 razy gorszy niż czas pojedynczego wątku procesora.

Co istotne, analiza dynamiki wzrostu czasu (Rysunek 2) wykazuje, że przyrost czasu przetwarzania dla platformy Hadoop charakteryzuje się znacznie większym współczynnikiem kierunkowym. Oznacza to, że czyste zwiększanie wolumenu danych w tym formacie nie doprowadzi do zrównania wyników. System Hadoop osiągnąłby przewagę (czyli realną wartość metryki COST) dopiero w scenariuszu, w którym rozmiar problemu przekroczyłby fizyczne możliwości pojedynczej maszyny – na przykład, gdyby zbiór danych nie mieścił się w pamięci RAM lub na dysku lokalnym węzła sekwencyjnego, wymuszając kosztowne operacje swapowania. W testowanym zakresie to algorytm sekwencyjny stanowi wzorzec wydajności.

7 Wnioski

- Wersja sekwencyjna okazała się lepsza od wersji równoległej pod każdym względem w badanym zakresie rozmiarów plików.
- Infrastruktura rozproszona Apache Hadoop generuje ogromny narzut. Narzut ten rośnie proporcjonalnie do rozmiaru danych, przez co czas wykonania klastra rósł liniowo w sposób znacznie bardziej stromy niż dla wersji sekwencyjnej.
- Wskaźnik przyspieszenia bezwzględnego oscylujący w granicach $S(N, p) \approx 0.11 - 0.13$, że użycie klastra obliczeniowego dla danych wielkości rzędu kilkunastu gigabajtów jest nieoptymalny.
- Porównanie architektur klastra wykazało nieznaczną, ale stałą przewagę skalowania pionowego (Konfiguracja B: 2 węzły **r8g.2xlarge**) nad skalowaniem poziomym (Konfiguracja A: 4 węzły **r8g.xlarge**).